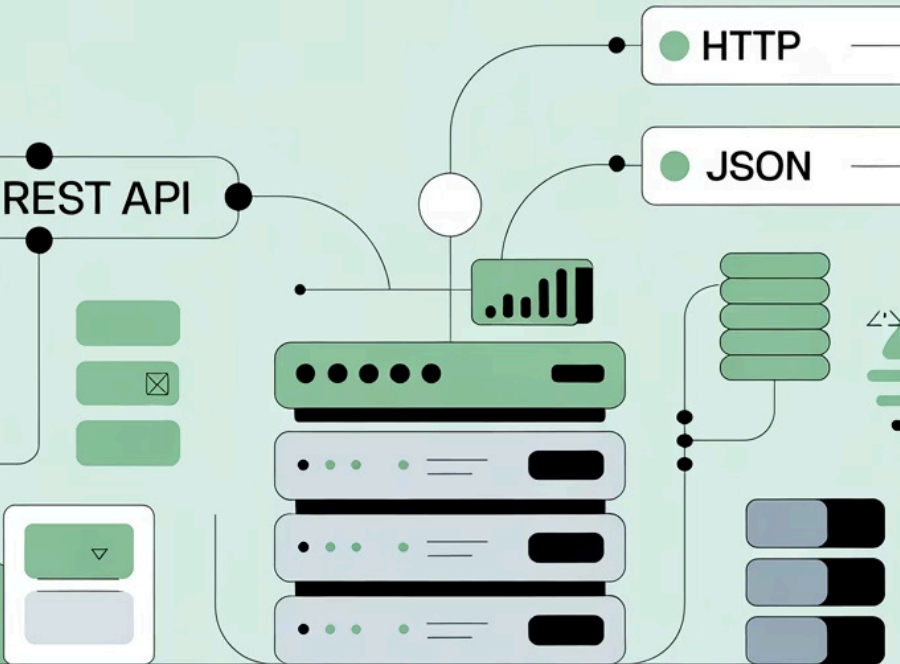


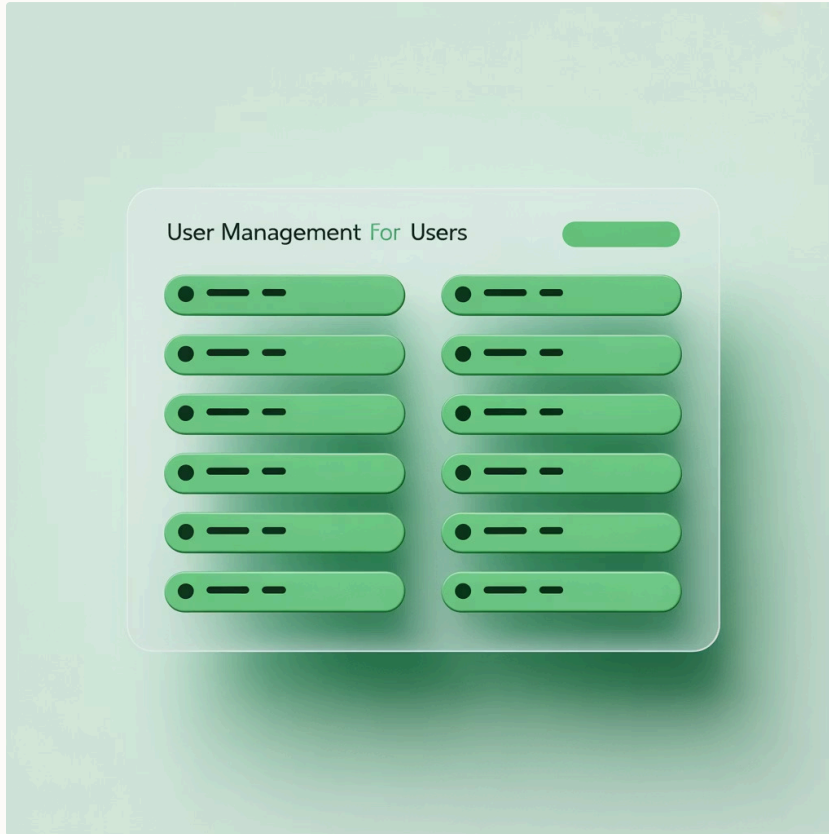


APIs REST con Spring Boot

De cero a un backend completo

Proyecto práctico: Sistema de Gestión de Usuarios

Nuestro Objetivo de Hoy



Lo que vamos a construir

- ☒ Backend completo con Spring Boot
- ☒ 4 endpoints REST funcionando
- ☒ Validaciones automáticas
- ☒ Arquitectura profesional en capas
- ☒ Persistencia en base de datos
- ☒ Documentación interactiva con Swagger

 Vamos a construir esto PASO A PASO durante la clase

¿Qué es una API REST?

Para construir un backend robusto, primero debemos entender los componentes fundamentales. Es importante diferenciar entre una API y el estilo arquitectónico REST.

¿Qué es una API?

Interfaz de Programación de Aplicaciones es un conjunto de reglas y protocolos que permiten que dos aplicaciones de software se comuniquen entre sí. Es como un "camarero" que toma tu pedido (solicitud) y lo lleva a la cocina (servidor), luego te trae la comida (respuesta).

- Facilita la integración entre diferentes sistemas.
- Abstrae la complejidad de la lógica interna del servicio.
- Define los puntos de entrada para interactuar con una aplicación.

En resumen, una **API REST** es una **API** que ha sido diseñada siguiendo los principios arquitectónicos de **REST**.

¿Qué es REST?

Representational State Transfer no es una tecnología, sino un estilo arquitectónico con un conjunto de principios para diseñar APIs web. Estas APIs son conocidas como APIs RESTful. Su objetivo es la escalabilidad y eficiencia.

- Se enfoca en **recursos** (datos) identificados por URIs únicas.
- Utiliza los métodos estándar de HTTP (GET, POST, PUT, DELETE) para operar sobre los recursos.
- Es "sin estado" (stateless): cada solicitud contiene toda la información necesaria.



Vamos a Excalidraw

Veamos cómo funciona la comunicación Cliente-Servidor
con HTTP

Dibujaremos juntos el flujo completo de una petición REST y cómo viaja la
información entre cliente y servidor.

Los 4 Métodos HTTP Esenciales

Estos métodos forman la base de cualquier API REST y representan las operaciones CRUD: **Create, Read, Update, Delete**.

MÉTODO	ACCIÓN	EJEMPLO	CÓDIGO RESPUESTA
 GET	Leer recursos	GET /usuarios GET /usuarios/5	200 OK
 POST	Crear nuevo recurso	POST /usuarios	201 Created
 PUT	Actualizar completo	PUT /usuarios/5	200 OK
 DELETE	Eliminar recurso	DELETE /usuarios/5	204 No Content

📌 Cada método tiene un propósito específico y códigos de respuesta estándar que indican el resultado de la operación.

@RestController - La Puerta de Entrada

¿Qué hace un RestController?

- Recibe peticiones HTTP del cliente
- Procesa la solicitud y coordina la lógica
- Devuelve respuestas automáticamente en formato JSON



Anotaciones clave de Spring



@RestController

Marca la clase como controlador REST que maneja peticiones HTTP



@RequestMapping

Define la ruta base para todos los endpoints del controlador



@GetMapping

Específico para peticiones GET de lectura



@PostMapping

Específico para peticiones POST de creación

Magia de Spring Boot: Convierte automáticamente objetos Java ↔ JSON sin código adicional



¡Manos a la obra!

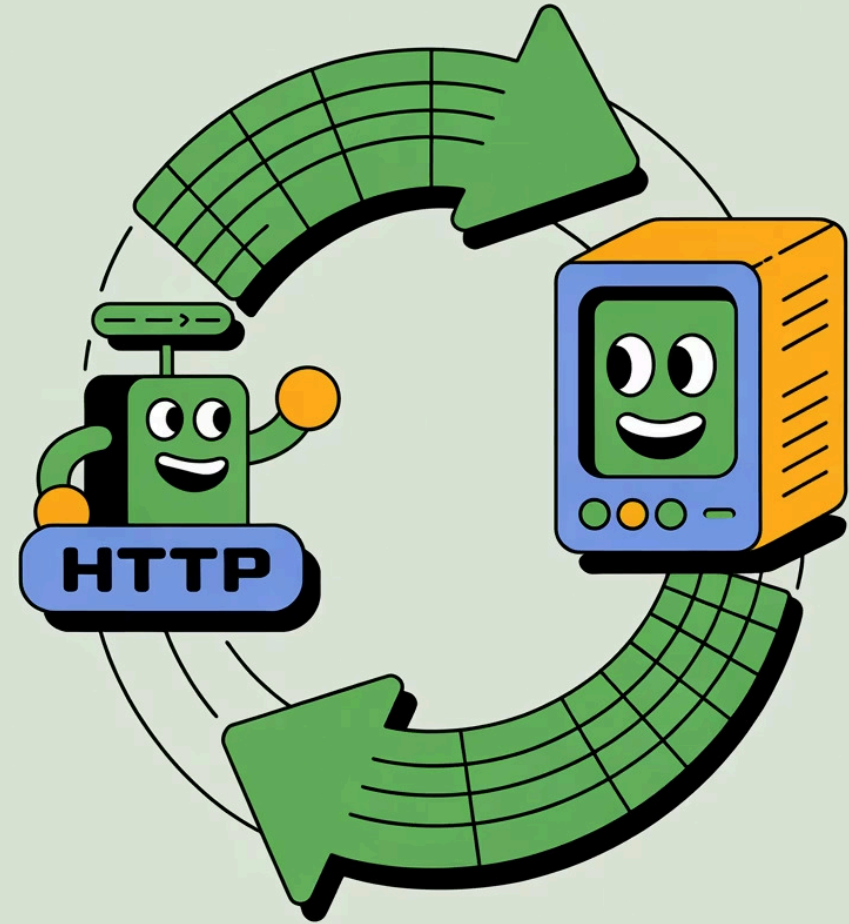
Creemos nuestro primer endpoint REST en Spring Boot

Vamos a codificar en vivo un controlador completo que responda a peticiones HTTP y devuelva datos en formato JSON.

Visualicemos los Métodos HTTP

Vamos a dibujar cómo cada método interactúa con
nuestros recursos

Entenderemos visualmente qué hace cada método, qué datos envía y qué respuestas
genera el servidor.





✗ Houston, tenemos un problema...

¿Qué pasa con código desorganizado?

- ✗ Lógica de negocio mezclada en el controlador
- ✗ Acceso a datos sin estructura clara
- ✗ Difícil o imposible de testear correctamente
- ✗ Mantenimiento se vuelve una pesadilla
- ✗ No escalable cuando crece el proyecto



📌 **Necesitamos ORGANIZACIÓN profesional** - Un archivo gigante con TODO mezclado es una receta para el desastre

✓ La Solución: Separación en Capas

La arquitectura en capas es un patrón de diseño que separa las responsabilidades de tu aplicación en módulos independientes. Cada capa tiene un propósito específico y bien definido.

🌐 @RestController

Capa de presentación: recibe peticiones HTTP y devuelve respuestas JSON

🧠 @Service

Capa de lógica de negocio: procesa las reglas y operaciones del dominio

💾 @Repository

Capa de acceso a datos: interactúa con la base de datos

🗄️ Base de Datos

Almacenamiento persistente de la información

Principio clave: Cada capa tiene UNA responsabilidad. Las capas se comunican mediante inyección de dependencias.

✓ Código ordenado

Fácil de navegar y mantener

✓ Testeable

Cada capa se prueba independientemente

✓ Cambios aislados

Modificar una capa no afecta las demás

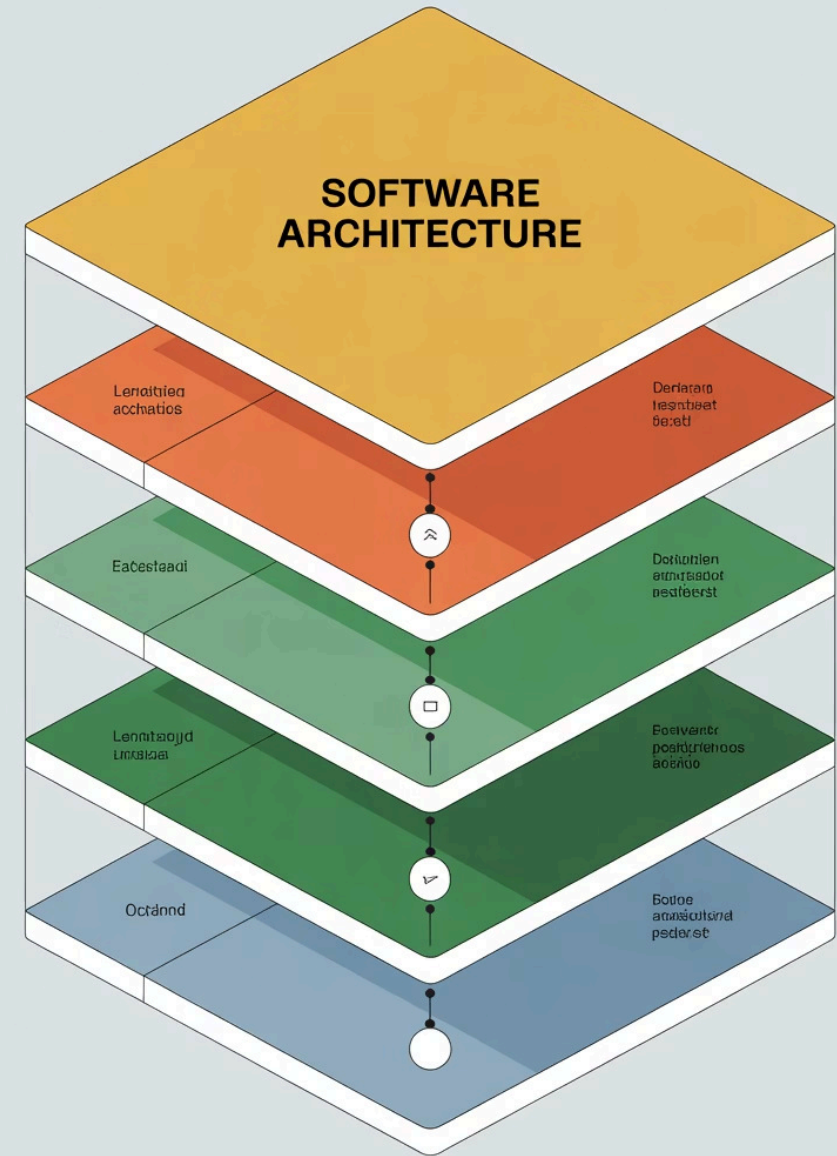
✓ Reutilizable

Servicios compartidos entre controladores

Dibujemos la Arquitectura Completa

Vamos a ver el flujo completo de una petición HTTP a través de las 3 capas

Trazaremos el camino que sigue una petición desde que llega al controlador hasta que se guarda en la base de datos y regresa como respuesta JSON.





Separaremos en Capas Nuestra Aplicación

Refactoricemos nuestro código para una arquitectura limpia

Ahora que tenemos nuestro primer endpoint funcionando, es momento de refactorizar y organizar nuestro código. Vamos a separar nuestra aplicación en las capas de Controller, Service y Repository para lograr una arquitectura profesional, escalable y mantenible.

Esto nos permitirá manejar la lógica de negocio, la comunicación con el cliente y el acceso a datos de manera independiente y eficiente.



DTOs: Data Transfer Objects

¿Por qué NO exponer la entidad directamente?

Riesgos de exponer entidades



Datos sensibles

Exponemos información confidencial como contraseñas o tokens



Acoplamiento

Frontend depende de la estructura interna de la base de datos



Sin control

No podemos validar ni transformar datos antes de enviarlos

Flujo con DTOs - Control total

1

Cliente envía datos

JSON con información del formulario

2

RequestDTO

Validamos y recibimos solo lo necesario

3

Entity (uso interno)

Convertimos a entidad para procesar

4

ResponseDTO

Filtramos qué exponemos al cliente

5

Cliente recibe respuesta

JSON seguro sin datos sensibles

Usuario (Entity)

- id
- nombre
- email
- password ⚠
- fechaCreacion

UsuarioResponseDTO

- id ✓
- nombre ✓
- email ✓
- ~~password~~ ✗
- ~~fechaCreacion~~ ✗

Los DTOs son FILTROS de seguridad y control - Protegen tu API y dan flexibilidad para cambiar la estructura interna sin afectar clientes

Java Moderno: Records + Validaciones

Records - Java 14+

Los **Records** son una forma concisa de crear clases inmutables para transportar datos. Eliminan código repetitivo (boilerplate).

✗ Antes (POJO)

- Constructor manual
- Getters y setters
- equals() y hashCode()
- toString()

30+ líneas de código

✓ Ahora (Record)

```
public record UsuarioDTO(  
    String nombre,  
    String email  
) {}
```

3 líneas ✨

Validaciones automáticas

Bean Validation (Jakarta) valida datos antes de procesarlos. Si hay errores, Spring devuelve automáticamente **400 Bad Request** con mensajes claros.



@NotBlank

El campo es obligatorio y no puede estar vacío



@Email

Valida que el formato sea un email válido



@Size(min=6)

Longitud mínima de caracteres requerida



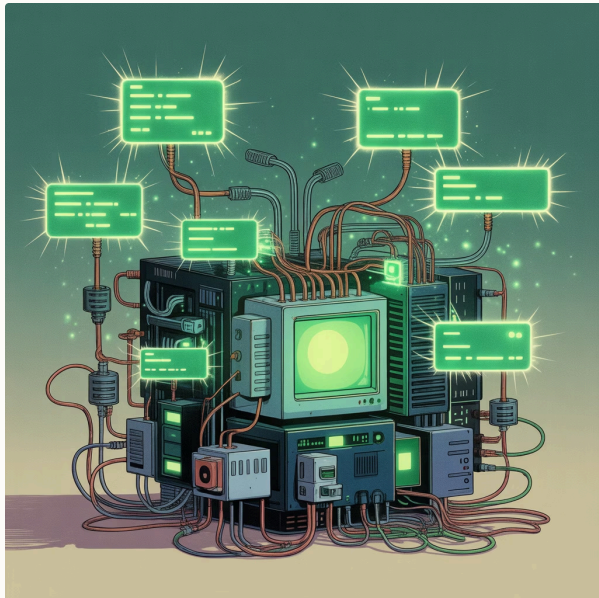
@Valid

Activa las validaciones en el método del controlador



Errores Profesionales con @ControllerAdvice

Problema sin manejo centralizado



- ❌ Errores diferentes según endpoint
- ❌ Stack traces internos expuestos al cliente
- ❌ Mensajes poco claros o técnicos
- ❌ Sin formato consistente

Solución con @ControllerAdvice



Error en cualquier Controller

Una excepción ocurre durante el procesamiento



@ControllerAdvice intercepta

Captura la excepción globalmente



ErrorDTO estructurado

Crea respuesta en formato consistente



Cliente recibe JSON claro

Respuesta legible y procesable

Estructura del ErrorDTO

```
{
  "timestamp": "2025-11-02T14:30:00",
  "status": 400,
  "error": "Error de validación",
  "detalles": [
    "nombre: no puede estar vacío",
    "email: formato de email inválido"
  ]
}
```

✓ Formato único

Toda la API devuelve errores con la misma estructura

✓ Seguridad

Sin datos internos ni stack traces expuestos

✓ Fácil consumo

Frontend puede procesar errores de forma consistente



Persistencia con Spring Data JPA + H2

Vamos a evolucionar nuestro proyecto desde almacenamiento en memoria (que se pierde al reiniciar) a persistencia real en base de datos.

❌ FASE 1 - Hasta ahora

Almacenamiento con `List<Usuario>` en memoria

Problema: Los datos se pierden cada vez que reiniciamos la aplicación

✅ FASE 2 - Ahora

Base de datos H2 embebida con Spring Data JPA

Ventaja: Los datos persisten y sobreviven reinicios del servidor

Cambios necesarios (minimalistas)

01

Agregar dependencias

Spring Data JPA + H2 Database en el `pom.xml`

03

Cambiar Repository

Extender `JpaRepository<Usuario, Long>`

02

Anotar la entidad

Usar `@Entity` y `@Table(name = "usuarios")`

04

Configurar properties

Añadir configuración de H2 en `application.properties`

Magia de Spring Data JPA: NO escribimos SQL manualmente. Los métodos CRUD son automáticos. Podemos crear consultas solo escribiendo el nombre del método: `findByEmail(String email)`

📌 **Importante:** Controller y Service NO necesitan cambios. Solo modificamos el Repository. Esto demuestra el poder de la arquitectura en capas.



Bonus: Documentación con Swagger

¿Qué es Swagger/OpenAPI?

Herramienta que genera **documentación interactiva automática** de tu API REST. Los desarrolladores pueden ver todos los endpoints, probarlos directamente desde el navegador y ver los esquemas de datos.

¿Cómo agregarlo?

1. Agregar una dependencia en `pom.xml`
2. Ejecutar la aplicación
3. Abrir `localhost:8080/swagger-ui/index.html`

Funcionalidades de Swagger UI



Lista de endpoints

Todos los controladores y métodos disponibles organizados



Botón "Try it out"

Ejecuta peticiones reales directamente desde el navegador



Esquemas de DTOs

Visualiza la estructura de datos de entrada y salida



Próximos Pasos en tu Aprendizaje

Ahora que dominas los fundamentos de APIs REST con Spring Boot, estos son los siguientes temas para llevar tus habilidades al siguiente nivel profesional.



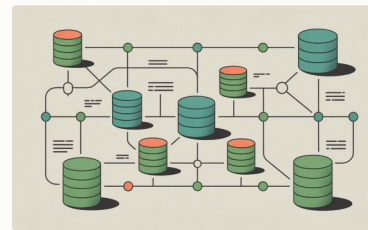
Seguridad con Spring Security

Implementa autenticación JWT, roles de usuario, protección de endpoints y autorización basada en permisos.



Paginación y Filtros

Maneja grandes volúmenes de datos con paginación eficiente, ordenamiento dinámico y filtros avanzados.



Relaciones entre Entidades

Domina @OneToMany, @ManyToOne, @ManyToMany y cómo JPA maneja las relaciones complejas entre tablas.



Deploy en la Nube

Despliega tu aplicación en plataformas como Render, Railway o AWS para que esté disponible 24/7.

Lo que Construimos Hoy



API REST completa

4 endpoints funcionales que cubren todas las operaciones CRUD



Arquitectura en capas

Código organizado profesionalmente en Controller, Service y Repository



Validaciones automáticas

Bean Validation protege tu API de datos incorrectos



Persistencia en BD

Spring Data JPA + H2 para almacenamiento real



Documentación Swagger

API autodocumentada y lista para ser consumida



Manejo de errores

@ControllerAdvice para respuestas consistentes



Ahora te toca practicar

Crea tu propia API REST

Elige un dominio que te apasione (libros, películas, productos, tareas) y construye tu propia API desde cero aplicando todo lo aprendido. La práctica es la clave para dominar Spring Boot.

Recuerda: El mejor código es el que escribes tú mismo. No tengas miedo de experimentar, equivocarte y aprender. Cada error te acerca más al dominio de Spring Boot.